

Change request log #3

1. Concept Location

Step #	Description	Rationale
1	Change request description analysis	Understood the scope and nature of the change request.
2	Found key concepts: Menu, Edit, Scroll Bar, Horizontal Scroll, Vertical Scroll	Identified crucial elements relevant to the change request for analysis and potential modification.
3	Additional concepts: buffer, view pane, ScrollBar, hide, disable, show, enable	Expanded the understanding of elements potentially impacted by the change request.
4	Interacted with jEdit: Menu > Edit; Menu > Utilities > Buffer Options, Global Options; Menu > View > Scrolling. Browsed packages: org.gjt.sp.jedit.menu, org.gjt.sp.jedit.options	Directly engaged with the jEdit interface to explore relevant functionalities and configurations. Examined source code structures to locate relevant components and configurations.
5	Text Search: "Word Count..." (as seen in Menu > Edit > Word Count...). Led to /jEdit/org/gjt/sp/jedit/jedit_gui.props	Conducted a specific search operation within the jEdit environment to locate relevant functionalities. Traced the source file associated with the identified functionality for potential modification or analysis. In the jedit_gui.props file the properties necessary for the addition of a new feature were found, so we marked this file as located.
6	Conducted text search and object search on "word-count:" in jEdit, leading to error in actions.xml	Employed search operations within jEdit environment to locate relevant functionality, leading to identification of error and need for further investigation. Searched specifically on "word-count:" and performed object search on 'TextArea' for targeted exploration. We marked the actions.xml file also as to be edited.
7	Then we searched using the keyword "TextArea.java"	Since we found the scroll bar in text area, we searched using this keyword and landed on the "TextArea.java" file. We inspected the file and found a method like updateScrollBar, we thought this method must be changed and marked as located.

Time spent (in minutes): 300 minutes.

Classes and methods inspected:

- build/classes/core/org/gjt/sp/jedit/jedit_gui.props
- /jEdit/org/gjt/sp/jedit/actions.xml
- jEdit/org/gjt/sp/jedit/textarea/JEditTextArea.java
 - updateScrollBar()

2. Impact Analysis

Step #	Description	Rationale
1	JEditTextArea.java/JEditTextArea: updateScrollBar() method	The absence of scroll bars in windows other than the text area suggested a potential issue with the updateScrollBar() method within the JEditTextArea class. Since, this method is responsible for managing the display and behavior of scroll bars within the text area component. We thought that analyzing the implementation and invocation of this method will provide insights into why scroll bars were not being rendered in other windows and guide efforts to rectify the issue. But on analyzing this function we didn't find anything useful, so we marked as not to be changed.
2	GUI-related classes and methods (e.g., dialog boxes, menu windows)	On analyzing the other methods, we concluded that the existing methods needn't be changed instead new methods had to be added to perform the necessary changes.

Time spent (in minutes): 100 minutes.

Classes and methods inspected:

- jEdit/org/gjt/sp/jedit/textarea/JEditTextArea.java
 - updateScrollBar()

3. Actualization

Step #	Description	Rationale
1	We first inspected the jedit_gui.props file to add the “hide scrollbar” and “show scrollbar” options in the dropdown menu of view.	In the jedit_gui.props, we made a change by adding the following 4 lines which defines properties related to scrollbars in jEdit. 1. disable-scrollbar.label=\$Hide Scrollbars: This line defines the label text for hiding the scrollbars. The \$Hide Scrollbars text will be displayed in the UI. 2. enable-scrollbar.label=\$Show Scrollbars: This line defines the label text for showing the scrollbars. The \$Show Scrollbars text will be displayed in the UI. 3. disable-scrollbar: This property represents configuration to hide the scrollbars when triggered. 4. enable-scrollbar: This property represents configuration to show the scrollbars when triggered.
2	We then made changes to the actions.xml file, this file would be responsible for specifying the necessary actions for the options provided in the drop-down menu	We added the code to specify the actions, for the options we added in the view menu. <pre><ACTION NAME="disable-scrollbar"> <CODE> textArea.disableScrollBars(); </CODE> </ACTION></pre> <pre><ACTION NAME="enable-scrollbar"> <CODE> textArea.enableScrollBars(); </CODE> </ACTION></pre>
3.	Finally we added two new methods “disableScrollBars()” in the TextArea.java file and added functionality	We added the <pre>public void disableScrollBars(){ scrollPane.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_NEVER); scrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_NEVER); }</pre> This code threw an error stating that JScrollPane was not found. So, we had to revisit the entire code base again to find what exactly the code used.
4.	We rectified our error and made change again to “TextArea.java”	While we re-checked the code base we were able to find methods such as vertical.setEnabled and horizontal.setEnabled, so decided to make the changes by adding the following code: <pre>public void disableScrollBars(){ vertical.setEnabled(false); horizontal.setEnabled(false); }</pre> And then for enable scrollbar we added the following code: <pre>public void enableScrollBars(){ vertical.setEnabled(true); horizontal.setEnabled(true); }</pre>

Time spent (in minutes): 250 minutes.

Classes Inspected

- build/classes/core/org/gjt/sp/jedit/jedit_gui.props
- /jEdit/org/gjt/sp/jedit/actions.xml
- JEdit/org/gjt/sp/jedit/textarea/JEditTextArea.java

Classes Changed and Methods Added

- build/classes/core/org/gjt/sp/jedit/jedit_gui.props
 - View menu
 - “Hide Scrollbar” item added.
 - “Show Scrollbar” item added.
- jEdit/org/gjt/sp/jedit/actions.xml
 - Enable scrollbar action added.
 - Disable scrollbar action added.
- JEdit/org/gjt/sp/jedit/textarea/JEditTextArea.java
 - disableScrollBars() added.
 - enableScrollBars() added.

4. Validation

Step #	Description	Rationale	Result
1	Unit test the JEditTextArea.java file that is responsible for toggling scrollbar visibility.	Ensure that the individual function responsible for toggling scrollbar visibility works correctly.	Pass - the function correctly toggles the scrollbar visibility.
2	Unit test the event listener for the menu item or checkbox.	Verify that the event listener responds appropriately to user actions and triggers the correct function to toggle scrollbar visibility.	Pass - the event listener behaves as expected
3	Regression test the entire editor functionality after implementing scrollbar visibility changes.	Ensure that the overall functionality of the text editor has not been adversely affected by the changes related to scrollbar visibility.	Pass - all existing features work as expected with the new scrollbar visibility toggling option.
4	Regression test the responsiveness of the application.	Confirm that the jEdit application remains responsive and doesn't suffer from performance issues after the scrollbar visibility changes.	Pass- as the application is still responsive and performs well.
5	System test the jEdit application with the scrollbar visibility option enabled.	Evaluate how well the entire application functions with the new scrollbar visibility option, including opening, editing, and saving documents.	Pass - the application performs as expected and all functionalities work seamlessly with the scrollbar visibility option.
6	System test the jEdit application with the scrollbar visibility option disabled.	Ensure that the application remains functional even with the scrollbar visibility turned off, including checking for any unexpected behaviors.	Pass - the application remains stable and functions correctly without the scrollbar visibility.
7	Integration test the interaction between the menu item/checkbox and the function toggling scrollbar visibility.	Confirm that the menu item or checkbox correctly triggers the function responsible for toggling scrollbar visibility and that the integration is smooth.	Pass - the integration between the UI element and the functionality is seamless.
8	Integration test the scrollbar visibility feature with other existing features in the jEdit application.	Ensure that the new scrollbar visibility feature integrates well with other existing features, such as syntax highlighting, search functionality, etc.	Pass - there are no conflicts or issues when using the scrollbar visibility feature alongside other features.

9	Acceptance test the scrollbar visibility feature with a focus on user experience.	Evaluate how easy can we find, enable, and disable the scrollbar visibility option.	Pass - we find the feature intuitive and it enhances the overall user experience.
10	Acceptance test the entire jEdit application with the scrollbar visibility feature enabled and whether the requirement is met properly	Confirm that the overall jEdit experience is improved with the new scrollbar visibility option and that the change asked is done.	Pass – change asked is done appropriately.

Time spent (in minutes): 100 minutes.

5. Summary of the change request

Phase	Time (minutes)	No. of classes inspected	No. of classes changed	No. of methods inspected	No. of methods changes
Concept location	300	5	3	2	2
Impact Analysis	100	3	3	2	2
Actualization	250	5	3	7	7
Verification	100	3	3	7	7
Total	750	5	3	7	7

6. Conclusion

For this change, concept location was very hard to exactly land on the method where the GUI components, including the scrollbars are initialized and added to the editor window. We later found the class responsible for the main editor window. Impact analysis was quite easy because there is very small impact because of this code change, and we had to double ensure that the change was implemented correctly. For this, we have used Junit testing. Actualization and change propagation took relatively more time because many times, we could not resolve the change as required. While performing changes, we experienced errors stating that JScrollPane was not found. So, we had to revisit the entire code base again to find what exactly the code used. It was difficult to find and write the correct logic to toggle the visibility of the horizontal and vertical scrollbars based on the user's selection. It also took time to learn the configuration properties and how to add them in Jedit. Overall, This took us a long time because we had to learn how Jedit text editor's scrolling and other features works.